

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

КУРСОВАЯ РАБОТА
по дисциплине
«Интернет технологии и сетевое ПО»

по теме:
РАЗРАБОТКА МИКРОСЕРВИСНОГО ПРИЛОЖЕНИЯ

Студент:
Группа ИКС-432

Н.С. Штерниис

Преподаватель:
Ст. преподаватель

А.В. Лошкарев

Новосибирск 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ИСПОЛЬЗУЕМЫХ ТЕХНОЛОГИЙ .	4
1.1 Микросервисная архитектура	4
1.2 REST API и FastAPI	4
1.3 gRPC и Protocol Buffers	6
1.4 Обратный прокси-сервер (Nginx)	6
1.5 WebRTC	7
1.6 Паттерны отказоустойчивости	7
1.7 Kubernetes и Helm	8
1.8 Docker и Docker Compose	8
2 АРХИТЕКТУРА И РЕАЛИЗАЦИЯ ПРОЕКТА	10
2.1 Общая архитектура	10
2.2 Описание Proto-контракта	10
2.3 Бэкенд-сервис (backend)	11
2.4 Отказоустойчивость: Retry, Circuit Breaker, Graceful Degradation	11
2.5 WebRTC синхронизация дашборда	12
2.6 Фронтенд	13
2.7 Оркестрация и развёртывание	13
3 РЕЗУЛЬТАТЫ РАБОТЫ	15
3.1 Проверка работоспособности	15
3.2 Маршрут одного запроса через систему	16
4 НЕПРЕРЫВНАЯ ИНТЕГРАЦИЯ (CI)	17
4.1 Основные понятия CI	17
4.2 GitHub Actions	17
4.3 Реализация CI-пайплайна	18
4.3.1 Структура пайплайна	18
ЗАКЛЮЧЕНИЕ	20
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	21

ВВЕДЕНИЕ

Современные веб-приложения редко представляют собой единый монолитный процесс. Промышленные системы строятся из множества независимых компонентов, каждый из которых выполняет строго определённую функцию и взаимодействует с остальными по чётко определённому протоколу. Такой подход называется **микросервисной архитектурой** и является одной из ключевых тем курса распределённых систем.

Цель данной курсовой работы - разработать распределённое веб-приложение, демонстрирующее принципы микросервисной архитектуры. Разрабатываемый проект представляет собой систему статистики товаров и лайков: при запуске база данных автоматически заполняется начальными товарами, а статический фронтенд отображает графики просмотров и лайков вместе со списком товаров.

В процессе работы были применены технологии и концепции, изученные на протяжении всего курса:

- разработка REST API на базе фреймворка FastAPI (Python);
- межсервисное взаимодействие по протоколу gRPC с использованием Protocol Buffers для сбора логов событий;
- контейнеризация каждого сервиса с помощью Docker;
- оркестрация контейнеров посредством Docker Compose;
- автоматизация проверок с помощью GitHub Actions.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ИСПОЛЬЗУЕМЫХ ТЕХНОЛОГИЙ

1.1 Микросервисная архитектура

Микросервисная архитектура - это подход к разработке программного обеспечения, при котором приложение строится как набор небольших, независимо развёртываемых сервисов. Каждый сервис отвечает за строго ограниченную бизнес-функцию и общается с другими сервисами через сеть [1].

Ключевые принципы микросервисной архитектуры:

- **Единственная ответственность** - каждый сервис делает одно дело и делает его хорошо;
- **Слабая связанность (loose coupling)** - падение одного сервиса не должно ломать остальные (отказоустойчивость);
- **Независимое развёртывание** - каждый сервис может быть обновлён без перезапуска всей системы;
- **Наблюдаемость (observability)** - система должна предоставлять логи и метрики для контроля своего состояния.

Противоположностью микросервисов является **монолитная архитектура**, где весь код приложения функционирует в одном процессе. Монолит проще запускать на начальном этапе, но он плохо масштабируется: при росте нагрузки приходится масштабировать всё приложение целиком.

1.2 REST API и FastAPI

REST (Representational State Transfer) - архитектурный стиль взаимодействия компонентов распределённого приложения в сети. REST использует стандартные HTTP-методы: GET (получить данные), POST (создать ресурс), PUT (обновить), DELETE (удалить) [2].

Основные принципы REST:

1. **Клиент-серверная модель** - разделение интерфейса от логики хранения данных;

2. **Отсутствие состояния (stateless)** - каждый запрос содержит всю необходимую информацию;
3. **Единообразие интерфейса** - стандартизированные URL и HTTP-методы.

FastAPI - современный Python-фреймворк для создания API [3]. Его ключевые особенности:

- автоматическая генерация интерактивной документации OpenAPI (Swagger UI) по аннотациям типов;
- валидация входных и выходных данных через библиотеку Pydantic;
- асинхронная обработка запросов через `async/await`;
- высокая производительность, сравнимая с Go и Node.js.

Пример описания Pydantic-модели для валидации выходных данных REST API в FastAPI:

```
class ProductOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    name: str
    category: str
    views: int
    likes: int
```

Пример реализации REST-эндпоинта для получения списка товаров на FastAPI:

```
@app.get("/api/products", response_model=List[ProductOut])
def list_products(db: Session = Depends(get_db)):
    products = db.query(ProductModel).all()
    _send_log("list_products", f"Запрошен список товаров ({len(products)})")
    return products
```

1.3 gRPC и Protocol Buffers

gRPC (Google Remote Procedure Call) - высокопроизводительный фреймворк для межсервисного взаимодействия. gRPC использует бинарный протокол передачи данных и работает поверх HTTP/2 [4].

Преимущества gRPC перед REST:

- **Скорость** - бинарная сериализация через Protocol Buffers значительно компактнее текстового JSON;
- **Строгая типизация** - контракт описывается в .proto-файле и компилируется в код;
- **Мультиплексирование** - множество запросов могут выполняться по одному TCP-соединению.

Protocol Buffers (protobuf) - язык описания интерфейсов и формат сериализации данных. Пример описания сервиса логирования в .proto-файле:

```
service LogService {  
    rpc SendLog (LogEntry) returns (LogResponse);  
    rpc GetLogs (GetLogsRequest) returns (LogList);  
}
```

Из этого описания утилита `protoc` генерирует Python-классы `LogServiceServicer` (серверная часть) и `LogServiceStub` (клиентская часть).

1.4 Обратный прокси-сервер (Nginx)

В распределенных системах для обеспечения единой точки входа и безопасности используется **обратный прокси-сервер** (Reverse Proxy). Он принимает внешние HTTP-запросы клиентов и перенаправляет их соответствующим внутренним службам.

В данном проекте функции обратного прокси-сервера выполняет веб-сервер **Nginx**. Он решает следующие задачи:

- раздача статических файлов фронтенда (HTML, CSS, JS);

- перенаправление запросов на API бэкенда (проксирование `/api/` на `backend:8000`);
- избавление от проблем CORS (Cross-Origin Resource Sharing) за счет предоставления единого порта входа для статики и API.

1.5 WebRTC

WebRTC (Web Real-Time Communication) - открытый стандарт для организации прямой одноранговой (P2P) связи между браузерами без промежуточного сервера. Он позволяет передавать медиа и произвольные данные в реальном времени с минимальной задержкой.

Ключевые компоненты WebRTC:

- **RTCPeerConnection** - управляет всем P2P-соединением, согласованием кодеков и шифрованием;
- **RTCDataChannel** - канал для передачи произвольных данных (текст, бинарные данные) между участниками без прохождения через сервер;
- **ICE (Interactive Connectivity Establishment)** - механизм обнаружения оптимального маршрута между участниками, включая обход NAT через STUN/TURN-серверы.

Для установки WebRTC-соединения требуется **сигнальный сервер** (в данном проекте реализован как WebSocket-эндпоинт бэкенда), через который происходит обмен SDP offer/answer и ICE-кандидатами. После установки соединения сигнальный сервер больше не задействован.

1.6 Паттерны отказоустойчивости

В распределённых системах отдельные сервисы могут быть временно недоступны. Для обеспечения устойчивости применяется несколько паттернов.

Retry (повторная попытка) - при сбое запрос автоматически повторяется через фиксированный или экспоненциально нарастающий интервал (*exponential backoff*). Позволяет пережить кратковременные сетевые сбои.

Circuit Breaker (предохранитель) - паттерн, предотвращающий каскадные отказы. При превышении порога ошибок «цепь размыкается» и дальнейшие попытки обращения к сервису прекращаются на фиксированное время. Это снимает нагрузку с нестабильного сервиса и устраняет накопление задержек в вышестоящих компонентах.

Graceful Degradation (плавная деградация) - при недоступности зависимого сервиса система не прекращает работу, а переходит в «пониженный» режим: ключевые функции продолжают работать, а необязательные (например, логирование) заменяются резервным поведением (запись в консоль).

1.7 Kubernetes и Helm

Kubernetes - система оркестрации контейнеров, которая автоматизирует развёртывание, масштабирование и управление жизненным циклом контейнеризованных приложений.

Ключевые концепции, использованные в проекте:

- **Deployment** - описание желаемого состояния набора подов с поддержкой RollingUpdate;
- **StatefulSet** - для PostgreSQL, обеспечивающий сохранность данных при перезапусках;
- **Job** - разовая задача инициализации базы данных;
- **RollingUpdate** со стратегией `maxSurge: 1, maxUnavailable: 0` - Zero Downtime обновление.

Helm - пакетный менеджер для Kubernetes. Позволяет описывать приложение в виде параметризованных шаблонов (*chart*) и управлять конфигурацией для разных окружений через файлы значений (`values-dev.yaml`, `values-stage.yaml`, `values-prod.yaml`).

1.8 Docker и Docker Compose

Docker - платформа для контейнеризации приложений. Контейнер представляет собой изолированную среду выполнения, которая содержит всё необходимое для запуска приложения: код, зависимости, конфигурацию.

Docker Compose - инструмент для описания и запуска многоконтейнерных приложений. Весь стек описывается в файле `docker-compose.yml` (сервисы, порты, тома, переменные окружения). Команда `docker compose up` -- собирает и запускает все сервисы одной командой, настраивая общую виртуальную сеть для их общения.

2 АРХИТЕКТУРА И РЕАЛИЗАЦИЯ ПРОЕКТА

2.1 Общая архитектура

Проект состоит из пяти контейнеризованных компонентов, представленных в **Таблице 1**.

Таблица 1 --- Сервисы приложения

Сервис	Технология	Функция
backend	Python + FastAPI	REST API, CRUD товаров, статистика, gRPC
log-svc	Python + gRPC	gRPC-сервер логирования, хранение логов в
db-init	Python + SQLAlchemy	Инициализация таблиц БД и заполнение нача
postgres	PostgreSQL	Хранение товаров, просмотров и лайков
frontend	Nginx + HTML/JS	Раздача статики, WebRTC DataChannel, интер

Схема взаимодействия: браузер подключается к Nginx на порту 3000. Nginx раздаёт статику и проксирует запросы `/api/*` и WebSocket `/ws/*` на бэкенд (`backend:8000`). Бэкенд работает с PostgreSQL и отправляет логи по gRPC в `log-svc:50051`. Между двумя браузерами устанавливается прямое P2P-соединение через WebRTC DataChannel, используя бэкенд только как сигнальный сервер.

2.2 Описание Proto-контракта

Контракт взаимодействия с сервисом логирования описан в файле `logs.proto`. Файл определяет сервис `LogService` со следующими RPC-методами:

- `SendLog` - отправка записи лога (`LogEntry`) на сервер;
- `GetLogs` - получение списка последних записей (`LogList`) с ограничением количества.

Компиляция proto-файла выполняется во время сборки Docker-образов:

```
python -m grpc_tools.protoc -I./proto \
```

```
--python_out=. --grpc_python_out=. \
proto/logs.proto
```

2.3 Бэкенд-сервис (backend)

Бэкенд реализован на FastAPI и предоставляет расширенный REST API:

- GET /health - проверка работоспособности;
- GET /api/products - список всех товаров;
- POST /api/products - ручное добавление товара (имя, категория, просмотры, лайки);
- PATCH /api/products/{id} - переименование товара;
- DELETE /api/products/{id} - удаление товара;
- PUT /api/products/{id}/popularity - обновление счётчиков просмотров и лайков;
- POST /api/products/{id}/like, DELETE /api/products/{id} - лайки;
- GET /api/stats - статистика (топ-7 по просмотрам и лайкам);
- GET /api/logs - получение логов из log-svc;
- WebSocket /ws/{room_id} - сигнальный сервер для WebRTC (SDP offer/answer, ICE).

2.4 Отказоустойчивость: Retry, Circuit Breaker, Graceful Degradation

Взаимодействие бэкенда с log-svc реализовано с тремя уровнями защиты от сбоев.

Circuit Breaker (предохранитель) реализован в виде класса `CircuitBreaker`. Он отслеживает количество последовательных ошибок: при достижении порога (5 ошибок) цепь размыкается (состояние OPEN) на 30 секунд, полностью прекращая сетевые обращения к log-svc и устраняя каскадные задержки. После периода ожидания автоматически выполняется пробный запрос (состояние HALF-OPEN).

Retry с экспоненциальной задержкой - при сбое каждая попытка отправки лога повторяется до 3 раз с паузами 0.5с, 1с и 2с (*exponential backoff*).

Graceful Degradation - при разомкнутом Circuit Breaker или исчерпании попыток Retry все основные API-эндпоинты (CRUD, статистика) продолжают работу в штатном режиме. Лог записывается локально в консоль бэкенда ([FALLBACK LOG]). Эндпоинт /api/logs возвращает HTTP 503 с понятным сообщением, а не падает с необработанной ошибкой.

Ключевой фрагмент реализации Circuit Breaker:

```
class CircuitBreaker:
    def allow_request(self):
        if self.state == "OPEN":
            if time.time() - self.last_state_change > self.recover_time:
                self.state = "HALF-OPEN"
                return True
            return False
        return True
```

2.5 WebRTC синхронизация дашборда

Для реализации совместного просмотра в реальном времени используется технология **WebRTC DataChannel** - прямое P2P-соединение между двумя браузерами без трафика через сервер.

Процесс установки соединения (сигналинг):

1. Первый пользователь открывает страницу и подключается к WebSocket /ws/dashboard-sync на бэкенде.
2. Второй пользователь подключается - бэкенд рассылает обоим участникам уведомление peer-joined.
3. Первый участник (caller) создаёт RTCDataChannel, формирует SDP offer и отправляет его через WebSocket-сигналинг.
4. Второй участник (callee) принимает offer, создаёт SDP answer, отправляет его обратно.
5. Происходит обмен ICE-кандидатами и устанавливается прямое P2P-соединение.

После установки соединения любое изменение (добавление/удаление/переименование товара, изменение фильтра) передаётся по DataChannel и немедленно применяется в браузере второго пользователя без перезагрузки страницы.

2.6 Фронтенд

Фронтенд - статическая HTML-страница с CSS и JavaScript. Включает:

- интерактивные графики Chart.js (топ-7 по просмотрам, топ-7 по лайкам);
- форму ручного добавления товара с полями имени, категории, просмотров и лайков;
- таблицу всех товаров с кнопками «Переименовать», «Популярность» и «Удалить»;
- индикатор статуса WebRTC (Offline / Connecting / Synced) с цветовым маркером;
- фильтрацию по категориям с мгновенным обновлением графиков.

Nginx в контейнере frontend раздаёт статику и проксирует HTTP-запросы на бэкенд, а также осуществляет WebSocket-проксирование с заголовками Upgrade и Connection: upgrade.

2.7 Оркестрация и развёртывание

Docker Compose используется для локальной разработки. Порядок запуска контейнеров управляется `depends_on` с `healthcheck` для PostgreSQL.

Для развёртывания в **Kubernetes** разработан Helm-чарт (директория `helm/`) с простой структурой по образцу `week-13`:

- `templates/deployment.yaml` - единый файл с описанием всех рабочих нагрузок (Secret, ConfigMap, StatefulSet для postgres, Deployment для log-svc/backend/frontend, Job для db-init);
- `templates/service.yaml` - все Kubernetes-сервисы;
- `values-dev.yaml`, `values-stage.yaml`, `values-prod.yaml` - конфигурации для разных стадий (количество реплик, лимиты ресурсов, теги образов).

Для сервисов backend и frontend настроена стратегия RollingUpdate с maxSurge: 1 и maxUnavailable: 0, что гарантирует Zero Downtime при обновлениях.

3 РЕЗУЛЬТАТЫ РАБОТЫ

3.1 Проверка работоспособности

После сборки и запуска проекта командой `docker compose up --build` все контейнеры переходят в состояние Up. Корректность работы системы проверялась следующими запросами:

```
-- GET      http://localhost:8000/health - возвращает
{"status": "ok"};
-- GET http://localhost:8000/api/products - возвращает
JSON-массив товаров;
-- POST http://localhost:8000/api/products - создаёт но-
вый товар с заданными параметрами;
-- PATCH http://localhost:8000/api/products/{id} - об-
новляет название товара;
-- DELETE http://localhost:8000/api/products/{id} -
удаляет товар;
-- PUT http://localhost:8000/api/products/{id}/popularity
- обновляет счётчики просмотров и лайков;
-- POST http://localhost:8000/api/products/1/like -
увеличивает лайки;
-- GET http://localhost:8000/api/stats - возвращает сум-
марную статистику и топ-7;
-- GET http://localhost:8000/api/logs - возвращает логи из
log-svc через gRPC;
-- GET http://localhost:3000/ - фронтенд с интерактивными
графиками через Nginx.
```

Для проверки WebRTC-синхронизации открывались два браузерных окна на `http://localhost:3000`. После установки P2P-соединения (статус «Synced» в обоих окнах) изменение данных в одном окне немедленно отображалось в другом без перезагрузки.

Для проверки Circuit Breaker был остановлен контейнер `log-svc`. Бэкенд после 5 ошибок переходил в состояние OPEN, в консоли появлялись

сообщения `[FALLBACK LOG]`, а все CRUD-операции с товарами продолжали работать без задержек. Через 30 секунд Circuit Breaker автоматически переходил в `HALF-OPEN` и восстанавливал соединение после перезапуска `log-svc`.

Helm-чарт валидировался командами:

```
helm lint weeks/week-17/starter/helm
helm template product-popularity weeks/week-17/starter/helm \
  -f weeks/week-17/starter/helm/values-dev.yaml
```

Оба шага завершились успешно без ошибок.

3.2 Маршрут одного запроса через систему

Для наглядности рассмотрим путь запроса на добавление нового товара:

1. Пользователь заполняет форму (имя, категория, просмотры, лайки) на веб-странице и нажимает кнопку «Добавить».
2. Браузер отправляет `POST /api/products` с JSON-телом на порт 3000.
3. Nginx проксирует запрос на `backend:8000`.
4. FastAPI-бэкенд создаёт запись в PostgreSQL через SQLAlchemy и возвращает созданный товар в формате JSON.
5. Бэкенд вызывает `_send_log()`, которая пытается отправить лог в `log-svc` (с `Retry + Circuit Breaker`). При успехе лог записывается в памяти `log-svc`; при сбое - в консоль бэкенда (`[FALLBACK LOG]`).
6. JavaScript получает ответ, обновляет таблицу и графики.
7. Если установлено WebRTC-соединение, бэкенд рассылает событие `{"type": "data-reload"}` всем участникам WebSocket-комнаты. Второй браузер получает событие по `DataChannel` и автоматически перезапрашивает список товаров.

4 НЕПРЕРЫВНАЯ ИНТЕГРАЦИЯ (CI)

4.1 Основные понятия CI

Непрерывная интеграция (Continuous Integration, CI) - это практика разработки программного обеспечения, при которой изменения кода автоматически проверяются, собираются и тестируются в единой целевой среде сразу после внесения в репозиторий.

Основная цель CI - раннее выявление проблем интеграции, когда несколько разработчиков работают над разными частями приложения. При каждом коммите (или создании pull request) CI-сервер автоматически запускает пайплайн (набор шагов), который подтверждает, что новые изменения не сломали существующий функционал. Если что-то пошло не так - команда получает оповещение и может оперативно исправить ошибку.

4.2 GitHub Actions

В качестве CI-платформы в данном проекте применяется **GitHub Actions** - инструмент автоматизации, встроенный непосредственно в GitHub. Конфигурация процессов (workflows) описывается в YAML-файлах, которые хранятся в папке `.github/workflows/` репозитория.

Базовые компоненты GitHub Actions:

- **Workflow** - общий автоматизированный процесс, настроенный в репозитории;
- **Trigger (on)** - событие, которое запускает процесс (например, push в определённую ветку или обновление файлов по определённому пути);
- **Job** - набор шагов (steps), выполняемых на одной виртуальной машине (runner);
- **Step** - конкретное действие, например, запуск консольной команды через `run` или вызов готового модуля через `uses`.

4.3 Реализация CI-пайплайна

Для проекта был разработан максимально простой, но эффективный CI-конвейер (`.github/workflows/week-17-simple.yml`). Основная задача этого пайплайна - гарантировать, что все контейнеры приложения могут успешно собраться и запуститься, а API корректно отвечает на базовые запросы.

Пайплайн запускается автоматически при каждом `push` или открытии `pull_request`, если были изменены файлы в директории проекта (`weeks/week-17/starter/`).

4.3.1 Структура пайплайна

Весь процесс состоит из одной `job`, которая выполняется на виртуальной машине с операционной системой Ubuntu (`ubuntu-latest`) и включает в себя следующие шаги:

1. **Клонирование репозитория:** Использование стандартного экшена `actions/checkout@v4` для получения исходного кода.
2. **Сборка и запуск контейнеров:** С помощью команды `docker compose up` собираются образы всех четырёх сервисов и запускаются контейнеры в фоновом режиме. Добавляется пауза в несколько секунд для полной инициализации сервисов.
3. **Проверка Health Check:** Выполнение запроса `curl` к эндпоинту `/health` API-шлюза для проверки его доступности.
4. **Интеграционное тестирование через curl:** Отправка тестовых запросов к эндпоинту `/api/products` (напрямую к шлюзу и через Nginx-фронтенд) с проверкой наличия ожидаемых полей (например, поля `id` в формате JSON).
5. **Сбор логов при ошибке:** Если какой-либо из предыдущих шагов завершается неудачно, с помощью условия `if: failure()` автоматически выводятся логи всех контейнеров для упрощения диагностики проблемы.
6. **Очистка ресурсов:** В самом конце, независимо от успешности или неуспешности выполнения тестов (используется условие

`if: always()`), контейнеры останавливаются и удаляются командой `docker compose down`.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы разработано полнофункциональное микросервисное приложение для мониторинга популярности товаров с расширенным набором возможностей.

В рамках работы были применены следующие технологии и концепции:

- **gRPC и Protocol Buffers** - для строго типизированного высокопроизводительного взаимодействия backend и log-svc;
- **FastAPI** - для реализации REST API с автодокументацией OpenAPI, CRUD-операциями и WebSocket-сигналингом;
- **WebRTC DataChannel** - для P2P-синхронизации состояния дашборда между несколькими пользователями в реальном времени без дополнительной нагрузки на сервер;
- **Docker и Docker Compose** - контейнеризация и оркестрация всех сервисов для локальной разработки;
- **Helm-чарт для Kubernetes** - описание развёртывания приложения в кластере со стратегией RollingUpdate (Zero Downtime) и тремя конфигурациями окружений (dev, stage, prod);
- **GitHub Actions CI** - автоматическая сборка и базовое интеграционное тестирование при каждом push.

Результат работы полностью функционален: система запускается командой `docker compose up --build`, автоматически заполняется данными и предоставляет веб-интерфейс с интерактивными графиками Chart.js, CRUD-управлением товарами и WebRTC-синхронизацией между пользователями.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Карабуртов Ю.* Просто о микросервисах. — 2018. — URL: <https://habr.com/ru/companies/raiffeisenbank/articles/346380/> (дата обр. 13.05.2026).
2. *Бураков А.* REST, что же ты такое? Понятное введение в технологию для ИТ-аналитиков. — 2021. — URL: <https://habr.com/ru/articles/590679/> (дата обр. 15.05.2026).
3. *Ramírez S.* FastAPI Documentation. — 2026. — URL: <https://fastapi.tiangolo.com/> (visited on 05/04/2026).
4. *Матевосян И.* Что нужно знать о gRPC системному аналитику. — 2023. — URL: <https://habr.com/ru/companies/tbank/articles/780024/> (дата обр. 20.05.2026).
5. *The PostgreSQL Global Development Group.* PostgreSQL Documentation. — 2026. — URL: <https://www.postgresql.org/docs/> (visited on 05/21/2026).
6. *Казаков А.* SQL в SQLAlchemy. — 2021. — URL: <https://habr.com/ru/companies/domclick/articles/581304/> (дата обр. 23.05.2026).